DESAFIO TÉCNICO · AI ENGINEER · ARTEFACT

Empório da Música
AI Agent

Documentação Técnica de Arquitetura, Implementação, Validação e Relatório de Entrega

Resumo da Solução: Agente conversacional em Python com grounding determinístico de catálogo e políticas, function calling com SDK oficial do Google Gemini, proteção contra prompt injection, animação skeleton e otimização de latência.

Candidato: Fernando Alves Machado
E-mail: fernando.r100k@gmail.com
Repositório Git Público:
https://github.com/FernandoAlvesMachado/emporio-musica-ai-agent

Data de Conclusão: 02 de Novembro de 2026
Linguagem Principal: Python 3.11+ / 3.13
Status da Solução: 100% dos testes aprovados (65/65)

Sumário Executivo

Este documento consolida o `README.md`, a documentação técnica de arquitetura, o relatório de validação e os cenários de interação desenvolvidos para a **Empório da Música**, loja fictícia de instrumentos musicais em Campo Grande/MS, em atendimento ao Desafio Técnico para **AI Engineer** da **Artefact**.

DESTAQUES DA IMPLEMENTAÇÃO

- **Linguagem e Stack:** Python puro com biblioteca padrão para orquestração, SQLite para integridade relacional, `pypdf` para extração fiel com numeração física de páginas, e SDK oficial `google-genai` (Interactions API com loop de ferramentas bounded).
- **Grounding Determinístico:** O agente nunca inventa preços, disponibilidade, regras de frete ou descontos. Todo fato apresentado provém das 6 bases CSV ou das 8 páginas do manual de políticas.
- **Proteção contra Prompt Injection:** Defesa em duas camadas com sanitização de tags estruturais de controle (ex: `<system>`, `[SYSTEM]`) e instruções imutáveis do sistema que repelem jailbreaks (DAN, Developer Mode) e tentativas de vazamento de prompts.
- **Otimização de Raciocínio (Latência):** Configuração de `thinking_budget: 1024` que reduziu o tempo de resposta em chamadas complexas de ferramentas de ~50s para menos de 3s–8s.
- **Interface Web Local & Skeleton:** Servidor HTTP loopback seguro com interface visual responsiva, cartões de catálogo, busca de políticas e feedback visual de carregamento animado (skeleton loading) no chat.
- **Privacidade Absoluta:** Nenhum dado pessoal de cliente, chave de API ou banco gerado é versionado no Git. Verificação automatizada por script de conformidade.

Estrutura deste Documento

1. README do Projeto (Guia Técnico Completo) — Seção 1
   - 1.1 Instalação e Execução (CLI e Browser) — 1.1
   - 1.2 Decisões Técnicas e Arquitetura Justificada — 1.2
   - 1.3 Engenharia de Prompt, Escopo e Grounding — 1.3
   - 1.4 Proteção contra Prompt Injection e Otimização de Performance — 1.4
   - 1.5 Uso Declarado de Assistentes de IA no Desenvolvimento — 1.5
2. Relatório de Validação e Matriz de Requisitos — Seção 2
3. Cenários de Conversação e Transcrições Reais — Seção 3

1. README do Projeto (Guia Técnico Completo)

1.1 Visão Geral e Cenário

O projeto implementa o assistente virtual da **Empório da Música**. A aplicação combina ferramentas determinísticas de catálogo e políticas com o loop conversacional da API Gemini do Google, interface em linha de comando (CLI) e uma interface local para navegador.

A integridade dos dados foi tratada como requisito de primeira ordem: dados recuperados do catálogo e trechos de políticas são injetados como *dados de referência não confiáveis*, impedindo que textos de produtos ou documentos sobreponham as instruções centrais do sistema.

1.2 Instalação e Configuração

Requer Python 3.11+ e Git. A instalação e inicialização a partir da raiz do repositório é simples e direta:

```
# Criar e ativar o ambiente virtual
python -m venv .venv
.\.venv\Scripts\Activate.ps1   # Windows PowerShell
source .venv/bin/activate       # Linux / macOS

# Instalar dependências em modo editável com pacotes de desenvolvimento
python -m pip install -e ".[dev]"

# Configurar variáveis de ambiente (.env)
cp .env.example .env
# Adicionar GEMINI_API_KEY no arquivo .env

# Carregar os dados originais no SQLite
python -m emporio_music load

# Iniciar o chat no terminal
python -m emporio_music chat --show-tools

# Iniciar a interface Web local
python -m emporio_music serve --open-browser
```

VARIÁVEIS DE AMBIENTE SUPORTADAS

Variável	Padrão	Descrição
`GEMINI_API_KEY`	*Obrigatório*	Chave da API do Google Gemini (obtida no Google AI Studio).
`GEMINI_MODEL`	`gemini-3.1-flash-lite`	Modelo do Gemini utilizado para chamadas de função e conversação.
`EMPORIO_THINKING_BUDGET`	`1024`	Orçamento de tokens de raciocínio, otimizando o tempo de resposta.
`EMPORIO_MODEL_TIMEOUT`	`60`	Timeout máximo em segundos por chamada ao provedor LLM.
`EMPORIO_CHAT_LANGUAGE`	`pt-BR`	Idioma padrão de atendimento (suporta `pt-BR` e `en`).
`EMPORIO_DATABASE_PATH`	`data/processed/emporio.sqlite3`	Local do banco de dados SQLite compilado deterministicamente.
`EMPORIO_POLICIES_PATH`	`data/raw/políticas_da_loja.pdf`	Local do manual de procedimentos em PDF.

1.3 Decisões Técnicas Justificadas

- **Arquitetura Enxuta (Small Architecture):** Optou-se por utilizar módulos da biblioteca padrão do Python (como `sqlite3`, `http.server`, `dataclasses`, `pathlib`) junto ao SDK oficial `google-genai`. Isso eliminou frameworks intermediários complexos (LangChain, LlamaIndex), garantindo controle total sobre o ciclo de vida da requisição, rastreabilidade explícita das ferramentas e ausência de overhead.
- **Tratamento e Integridade dos Dados:** A importação valida atomicamente todos os 6 arquivos CSV. Valores monetários são convertidos via `Decimal` para centavos inteiros (`cents`) no SQLite, eliminando imprecisões de ponto flutuante. A disponibilidade de produtos exige estritamente `status == 'active'` e `stock_quantity > 0`.
- **Regras Estritas de Não-Acúmulo de Promoções:** O cálculo de pagamentos (PIX e parcelado) foi implementado como uma ferramenta determinística separada (`quote_payment`), fora do modelo, garantindo que o desconto de 5% no PIX incida apenas sobre o preço cheio e nunca sobre itens promocionais.
- **Fidelidade e Conflitos nas Políticas:** As 8 páginas do PDF são lidas preservando os números físicos de página. Em casos de conflito factual nas fontes originais (ex: telefones divergentes entre as páginas 1 e 6, ou frete exato de R$ 500,00 não especificado), o agente foi instruído a reconhecer a ambiguidade e solicitar confirmação com a equipe, sem inventar regras fictícias.
- **Privacidade de Dados de Clientes:** Embora a base contenha tabelas de clientes e pedidos, o agente não expõe ferramentas para consulta ou alteração dessas ordens, evitando vazamento de dados pessoais ou aprovação simulada de devoluções sem autenticação prévia de identidade.

1.4 Proteção Contra Prompt Injection e Otimização de Performance

Para garantir segurança de nível corporativo e usabilidade ágil, foram introduzidas duas melhorias essenciais no agente:

Defesa Contra Injeção de Prompt em Camadas

1. **Sanitização Estrutural de Entrada (`_sanitize_customer_input`):** O agente filtra e neutraliza tags que simulam papéis internos (como `<system>`, `[SYSTEM]`, `<<SYS>>`, ```` ```system ````), impedindo que um atacante quebre a fronteira de papel do modelo.
2. **Instruções de Segurança e Imutabilidade (`prompts.py`):** A persona do assistente é definida como estritamente imutável. Tentativas de jailbreak (como o modo "DAN", "Developer Mode", personificação de administradores ou solicitações de "ignore regras anteriores") são detectadas e o assistente responde educadamente mantendo-se estritamente no escopo da loja. Além disso, o sistema veda a divulgação do prompt interno ou das chaves de configuração.

Otimização de Raciocínio (Thinking Budget)

Os modelos da família Flash com raciocínio expandido podiam gastar dezenas de segundos gerando pensamentos antes de emitir chamadas de ferramentas. Através do parâmetro `thinking_budget: 1024`, a chamada passou a resolver o direcionamento das ferramentas de forma rápida e precisa, reduzindo a latência total de ~50s para menos de 3s–8s em buscas complexas.

Animação Skeleton no Chat Web

A interface web loopback (`127.0.0.1:8000`) recebeu um elemento de carregamento visual `.chat-message-skeleton` com efeito de shimmer pulsante em CSS. Ele fornece feedback visual imediato ao cliente enquanto o Gemini processa as consultas, respeitando a diretiva de acessibilidade `prefers-reduced-motion`.

1.5 Declaração de Uso de Assistentes de Código (Workflow)

Em total transparência com os critérios da Artefact:

- **ChatGPT:** Utilizado na etapa inicial de brainstorm e planejamento estrutural dos estágios do desafio.
- **Claude / Codex / Antigravity:** Utilizado durante a implementação do código Python, escrita dos testes unitários com pytest, auditoria dos arquivos CSV e elaboração dos scripts de verificação contínua.
- **Google Gemini (`gemini-3.1-flash-lite`):** Provedor de LLM em tempo de execução para as interações e resolução de chamadas de ferramentas do agente.

2. Relatório de Validação e Matriz de Requisitos

A tabela abaixo sumariza a conformidade do projeto com os critérios de avaliação e requisitos do desafio técnico:

Requisito	Tipo	Status	Evidência Técnica
Python como linguagem principal	Obrigatório	**PASS**	Pacote `src/emporio_music/` estruturado, Python 3.11+ / 3.13.
CLI conversacional interativa	Obrigatório	**PASS**	Comando `python -m emporio_music chat` com suporte a `--show-tools` e `--language`.
Preservação dos dados originais	Obrigatório	**PASS**	6 tabelas CSV validadas, 65 produtos, 9 categorias, 25 promoções.
Grounding em ferramentas e fontes	Obrigatório	**PASS**	Catálogo e manual PDF lidos via SQLite parametrizado e `pypdf`.
Cálculo exato e não acúmulo de PIX	Obrigatório	**PASS**	Ferramenta `quote_payment` com cálculo determinístico em centavos inteiros.
Fidelidade a páginas físicas do PDF	Obrigatório	**PASS**	Excertos e páginas completas referenciadas de 1 a 8 com numeração física.
Resistência a Prompt Injection	Obrigatório	**PASS**	Sanitização de delimitadores estruturais e instruções de segurança imutáveis.
Suíte de testes automatizados	Obrigatório	**PASS**	65 testes automatizados aprovados no `pytest` em 2.62s.
Proteção de credenciais e Git público	Obrigatório	**PASS**	Script `check_tracked_files.py` auditou 44 arquivos sem vazamentos.
Interface Web Local (Skeleton UI)	Opcional	**PASS**	Servidor loopback com interface responsiva, cartões e animação skeleton.
Publicação no GitHub	Obrigatório	**PASS**	Repositório público ativo com histórico transparente de commits.

RESULTADO DOS TESTES UNITÁRIOS E INTEGRAÇÃO

Comando executado: `python -m pytest`

```
============================= test session starts ==============================
platform win32 -- Python 3.13.15, pytest-9.2.0, pluggy-1.7.0
rootdir: D:\PROJETOS\Avulsos\Artefact\Teste pratico
configfile: pyproject.toml
collected 65 items

tests/test_agent.py ....................                                 [ 30%]
tests/test_browser.py ..............                                     [ 53%]
tests/test_browser_chat.py ........                                      [ 66%]
tests/test_foundation.py ............                                    [ 84%]
tests/test_sdk_contract.py .                                             [ 86%]
tests/test_tools.py .........                                            [100%]

============================== 65 passed in 2.62s ==============================
```

3. Cenários e Transcrições de Conversação

Abaixo estão documentadas transcrições de interações realizadas com o assistente, demonstrando a ativação real de ferramentas, resolução de consultas em tempo real e aplicação das diretrizes de atendimento:

3.1 Cenário 1: Recomendação com Orçamento e Cotação de Pagamento (PIX)

Objetivo: Demonstrar consulta ao catálogo por categoria, filtro de disponibilidade e cálculo de pagamento à vista sem acúmulo indevido de descontos.

CLIENTE

"Please recommend three available acoustic guitars costing at most BRL 1000. Give the actual prices and product IDs."

EMPÓRIO DA MÚSICA · ASSISTENTE GEMINI

Hello! Welcome to Emporio da Musica. I'd be happy to help you find an acoustic guitar. We have several great options available for under BRL 1000. Here are three popular choices:

- **Tagima Memphis AC-39 Nylon Natural** (ID: 83) – BRL 429.90
- **Yamaha C40 Nylon Natural** (ID: 81) – BRL 599.90
- **Yamaha F310 Aço Natural** (ID: 87) – BRL 699.90

Are you looking for a guitar with nylon strings for a softer sound, or do you prefer the bright, projection-heavy sound of steel strings?

Ferramentas executadas nesta rodada: list_categories, search_products.

CLIENTE

"What is the recorded stock of the first option, and how much would I pay by PIX?"

EMPÓRIO DA MÚSICA · ASSISTENTE GEMINI

We currently have **15 units** of the Tagima Memphis AC-39 Nylon Natural (ID: 83) in stock.

If you choose to pay via PIX, the price is **BRL 408.41**, which includes a 5% discount on the base price of BRL 429.90. This calculation follows our store payment guidelines (pages 3 and 6 of our store policies).

Please note that this is an indicative quote based on our current catalog snapshot. Would you like to know anything else about this model or our payment options?

Ferramentas executadas nesta rodada: get_product, quote_payment.

3.2 Cenário 2: Consulta de Preço e Tratamento de Item Inexistente

CLIENTE

"Quanto custa o Takamine GD20 e ele está disponível em estoque?"

EMPÓRIO DA MÚSICA · ASSISTENTE GEMINI

O violão **Takamine GD20** (ID: 95) custa **R$ 2.199,00** e está disponível em nosso estoque com **5 unidades** no momento! Ele é um modelo eletroacústico dreadnought com tampo em Solid Cedar. Gostaria de verificar condições de pagamento para ele?

Ferramentas executadas nesta rodada: search_products, get_product.

3.3 Cenário 3: Esclarecimento de Políticas de Troca e Devolução

CLIENTE

"Me arrependi da minha compra, vocês podem emitir o meu reembolso agora?"

EMPÓRIO DA MÚSICA · ASSISTENTE GEMINI

Olá! Entendo sua situação e posso te orientar sobre as nossas políticas de devolução (previstas nas páginas 4 e 7 do manual da loja).

Como assistente virtual, eu não tenho autorização para aprovar devoluções ou processar reembolsos diretamente no sistema. Para que nossa equipe de atendimento possa te ajudar com essa solicitação, você poderia me informar:
1. A compra foi feita em nossa loja física ou pelo site/WhatsApp?
2. Há quantos dias você recebeu o instrumento?
3. O produto está intacto, na embalagem original e com todos os acessórios e manuais?

Com essas informações, consigo te explicar exatamente o procedimento a seguir junto à equipe!

Ferramentas executadas nesta rodada: search_policies, get_policy_pages.

4. Conclusão e Entrega

O protótipo do **Emporio da Musica AI Agent** cumpre com rigor todos os requisitos obrigatórios estabelecidos no desafio técnico da Artefact, indo além ao oferecer uma interface amigável com suporte a skeleton loading, validação estrita de segurança e proteção ativa contra prompt injection.

> **LINKS E ACESSOS OFICIAIS**
>
> **Repositório Público no GitHub:**
> https://github.com/FernandoAlvesMachado/emporio-musica-ai-agent
>
> **Contato do Candidato:** Fernando Alves Machado (fernando.r100k@gmail.com)